

Module 3

Introduction to Solidity

What Is Solidity?

The programming language for Ethereum smart contracts

You write Solidity → Compiler → EVM bytecode

Statically typed and contract-oriented

Why Solidity?

Easy to learn for developers

Designed specifically for smart contracts

Rich ecosystem of tools and libraries

Compilation Process:

Solidity source (.sol) → Compiler → EVM bytecode

Then deployed to Ethereum blockchain

File Structure

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract MyContract {
    // Your code here
}
```

License declaration

Required by tooling

Pragma sets compiler version

^ = 0.8.20 through 0.8.x

Never use floating pragma

(like >=0.8.0)

Contract Structure

```
// SPDX-License-Identifier: MIT
```

```
pragma solidity ^0.8.20;
```

```
contract MyContract {  
    // State variables  
    uint256 public data;  
    // Functions  
    function doSomething() public { }  
}
```

State Variables (Persistent Storage)

```
contract Counter {  
  
    uint256 public count;  
    address public owner;  
    bool public active;  
}
```

Stored permanently

in storage (on-chain)

public = auto-generates

getter function

Reading: cheap

Writing: expensive

(SSTORE ~20k gas)

Functions (Behaviour)

```
contract Counter {  
    uint256 public count;  
  
    function increment()  
        public  
    {  
        count += 1;  
    }  
}
```

Functions define behaviour

This function modifies state

count += 1 executes:

1. SLOAD (load storage)
2. ADD (add 1)
3. SSTORE (save storage)

SSTORE is most expensive

Function Visibility Rules

Visibility	Who Can Call?
public	Anyone (external + internal)
external	Only external callers
internal	Only this contract
private	Only inside this contract

View and Pure Functions

```
function getCount()  
    public view  
    returns (uint256)  
{  
    return count;  
}  
  
function calculateSum(uint256 a)  
    public pure  
    returns (uint256)  
{  
    return a + 1;  
}
```

view = read state, no modify

pure = no read, no modify

Off-chain calls: FREE

On-chain calls: pay gas

(Still cheaper than non-view)

Common Data Types

Type	Example	Size
uint256	0 to 2^256-1	256 bits
int256	-2^255 to 2^255-1	256 bits
bool	true/false	8 bits
address	0x123...	160 bits
string	"hello"	dynamic
bytes	0x1a2b	dynamic

Mappings (Key-Value Stores)

```
mapping(address => uint256)
    public balances;

mapping(uint256 => User)
    public users;
```

Dictionary in Python / Hashmap in Java

Key → Value lookup ($O(1)$)

Cannot iterate keys

Default value if missing

Lives in storage

(Expensive to modify)

Structs (Group Related Data)

```
struct User {  
    uint256 balance;  
    address wallet;  
    bool active;  
}  
  
mapping(address => User)  
public users;
```

Group multiple fields

together into one type

Create custom data types

Useful for complex
data modeling

Events (Off-Chain Notifications)

```
event Deposit(  
    address indexed user,  
    uint256 amount  
);  
  
function deposit() public payable {  
    emit Deposit(  
        msg.sender,  
        msg.value  
    );  
}
```

Emit logs to tx receipt

Off-chain systems listen

indexed = searchable

by filter

Critical for frontends

& indexers

Modifiers (Reusable Rules)

```
modifier onlyOwner() {  
    require(msg.sender  
        == owner);  
    _;  
}  
  
function withdraw()  
    public onlyOwner  
{  
    // function body  
}
```

`_` = where function runs

Enforce rules before executing

Reusable access control

Heavy use in security-critical code

Function Visibility Hierarchy



Putting It All Together

```
pragma solidity ^0.8.20;

contract SimpleBank {
    address public owner;
    mapping(address => uint256)
        public balances;

    event Deposit(address user,
                  uint256 amount);

    modifier onlyOwner() {
        require(msg.sender == owner);
        _;
    }

    constructor() {
        owner = msg.sender;
    }

    function deposit()
        public payable {
        balances[msg.sender]
            += msg.value;
        emit Deposit(msg.sender,
                     msg.value);
    }
}
```

License & pragma

State variables

Events emitted

Access control

Constructor runs once
at deployment



Security Considerations

Always Apply Access Control

Use modifiers to protect functions

Check msg.sender when needed

Be Careful with Storage Writes

SSTORE is expensive

Batch updates when possible

Events Are Critical

Frontend needs them to react

Index important parameters

Watch for Reentrancy

Especially with external calls

(More on this in Module 5)

Module 3 Recap: Solidity Basics

- ✓ Solidity is the language for Ethereum contracts
- ✓ State variables stored permanently in storage
- ✓ Functions define behaviour with visibility rules
- ✓ view functions don't modify state (cheaper)
- ✓ Mappings are key-value lookup tables
- ✓ Structs group related data together
- ✓ Events emit logs for off-chain systems
- ✓ Modifiers enforce reusable rules
- ✓ Storage writes are the most expensive

Key Insight:

Think about gas: minimise storage writes